

Supervised Learning: Classification

Classification in machine learning is a type of supervised learning where the goal is to categorize or classify data points into predefined classes or labels.

Given input data (features), a classification algorithm learns a **mapping function** that assigns a label (or class) to each data point based on the training data. It is used when the output variable is categorical, meaning it falls into one of several distinct categories.

Examples of Classification Algorithms:

Logistic Regression

K-Nearest Neighbors (KNN)

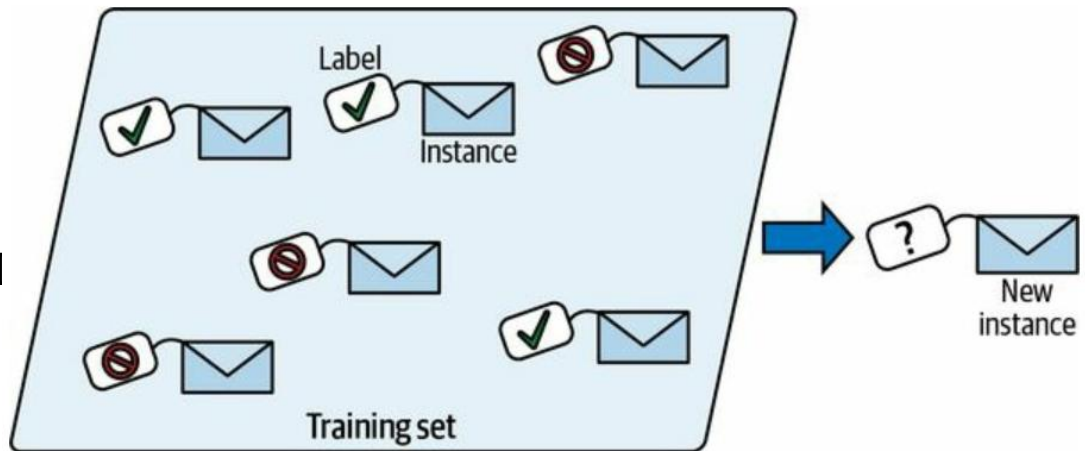
Decision Trees

Naive Bayes

Support Vector Machine (SVM)

Random Forest

Neural Networks



Classification : KNN

K-Nearest Neighbors (KNN)

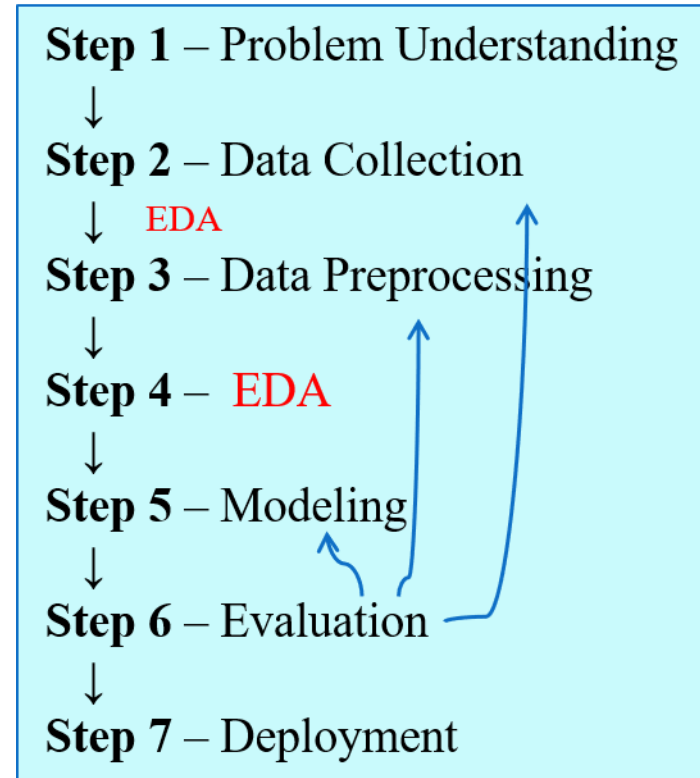
It is a simple, non-parametric, **instance-based** machine learning algorithm. It can be also used as regression tasks.

KNN works by finding the "k" closest data points (neighbors) to a new, unknown data point and making predictions based on their classes.

KNN **does not build a model** during the training phase. Instead, it stores the training data and makes predictions by referencing it directly during testing. This is why it is also called a **lazy learner**.

Advantages of KNN:

KNN is easy to understand and implement. Since KNN doesn't build a model, the training phase is **fast**. It can be used for both classification and regression problems. KNN doesn't make assumptions about the underlying data distribution (Non-parametric).



Classification : KNN

How KNN Works:

Select k Neighbors: The parameter "k" refers to the number of nearest neighbors to consider when making a prediction.

Calculate Distance: The algorithm calculates the distance between the new data point and all other points in the training dataset. Common distance metrics include: Euclidean Distance, Manhattan Distance.

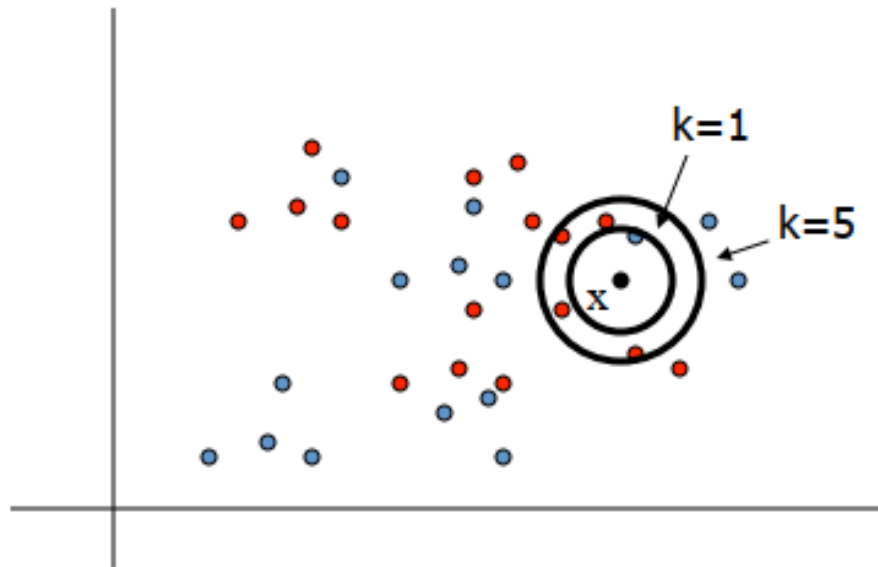
Identify Nearest Neighbors: After calculating the distances, the algorithm selects the k nearest points (neighbors) from the training set.

Make a Prediction: Classification: The new point is assigned to the class that is most common among its k nearest neighbors (majority voting).

Classification : KNN

How KNN Works:

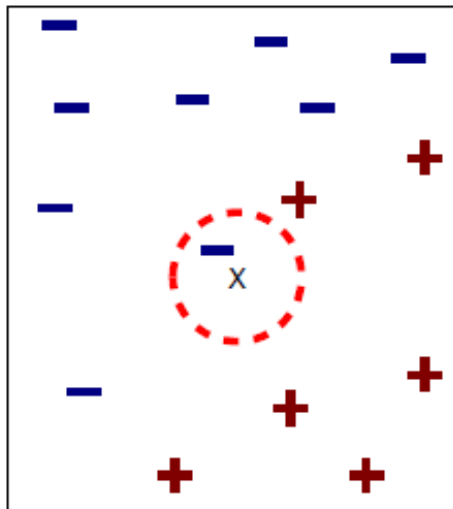
To classify a new input vector x , examine the k -closest training data points to x and assign the object to the most frequently occurring class.



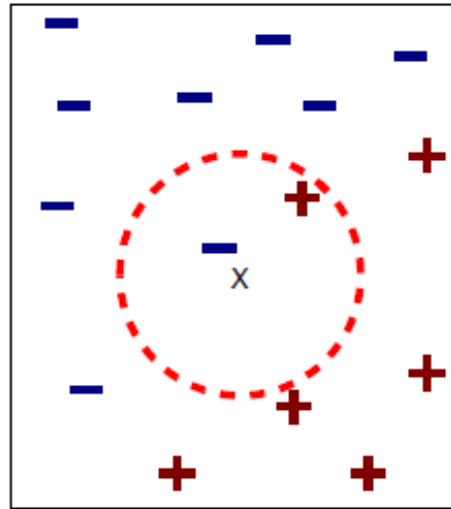
common values for k : 3, 5

Classification : KNN

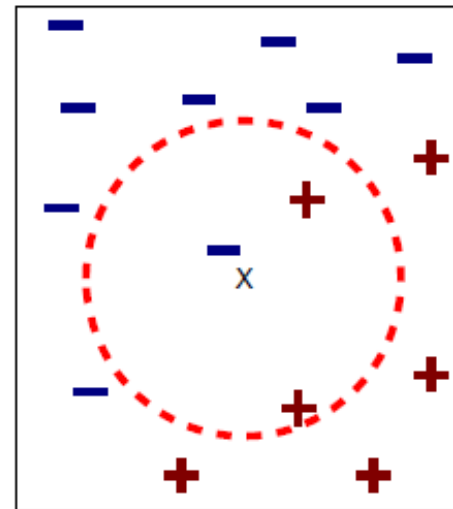
How KNN Works:



(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

Classification : KNN

Example of KNN:

Height (cm)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?

Classification : KNN

Example of KNN:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d_1 = \sqrt{(170 - 167)^2 + (57 - 51)^2}$$

$$d_2 = \sqrt{(170 - 182)^2 + (57 - 62)^2}$$

.

.

.

$$d_9 = \sqrt{(170 - 170)^2 + (57 - 55)^2}$$

Height (cm)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?

Classification : KNN

Example of KNN:

Height (cm)	Weight (KG)	Class	d
167	51	Underweight	6.7
182	62	Normal	13
176	69	Normal	13.4
173	64	Normal	7.6
172	65	Normal	8.2
174	56	Underweight	4.1
169	58	Normal	1.4
173	57	Normal	3
170	55	Normal	2
170	57	?	

Classification : KNN

Example of KNN:

Height (cm)	Weight (KG)	Class	d	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9
170	57	?		

Classification : KNN

Example of KNN:

Height (cm)	Weight (KG)	Class	d	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9
170	57	?		

} K=3

Classification : KNN

Example of KNN:

Height (cm)	Weight (KG)	Class	d	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9
170	57	?		

} K=4

Classification : KNN

Example of KNN:

Height (cm)	Weight (KG)	Class	d	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9
170	57	?		

K=5

Classification : KNN

KNN Code:

<https://colab.research.google.com/drive/1WvzjLHrb2Yf0ocPfUPUAIL7lYTIAvGwl?usp=sharing>

Classification : KNN

Disadvantages of KNN:

Computationally Expensive: As the size of the dataset increases, the algorithm becomes slow because it has to calculate the distance for each test point with every training point.

Sensitive to Noisy Data: Outliers or irrelevant features can significantly affect predictions.

Feature Scaling Required: Since KNN relies on distance calculations, features must be scaled properly to prevent features with larger ranges from dominating the distance calculation.

Memory-Intensive: The algorithm needs to store the entire training dataset, making it less memory-efficient for large datasets.

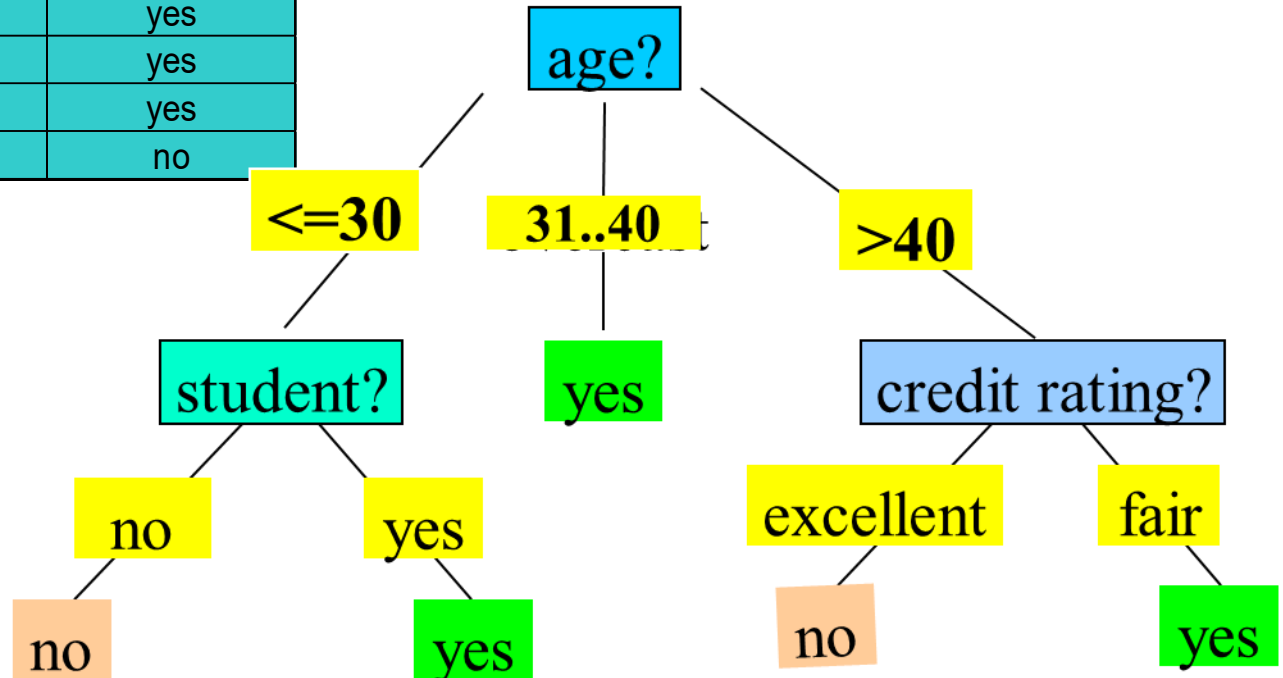
Classification : Decision Tree

Decision Trees are a popular supervised machine learning algorithm used for both **classification** and regression tasks. They work by learning simple decision rules inferred from the data features to predict a target label.

- A Decision Tree is structured as a tree where each node represents a decision based on a feature of the data, and each branch represents an outcome of that decision.
- The root node is the topmost node, representing the first decision.
- The internal nodes are decision points, splitting based on feature values.
- Leaf nodes are the endpoints representing the final prediction or output.

Classification : Decision Tree

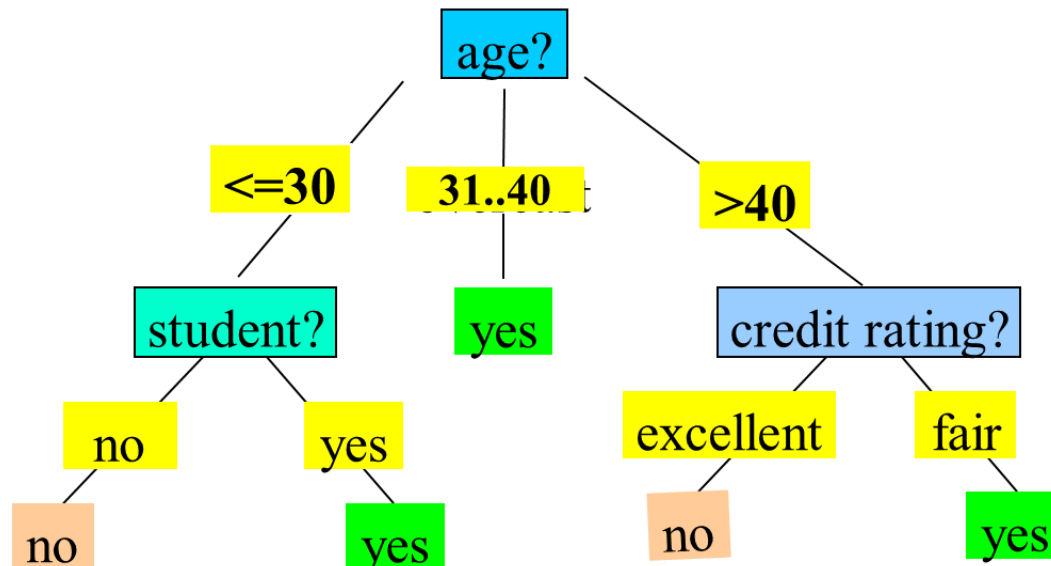
age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no



Classification : Decision Tree

Basic algorithm (a greedy algorithm):

- Tree is constructed in a **top-down recursive divide-and-conquer** manner
- At start, all the training examples are at the root
- Attributes are categorical (if continuous-valued, they are discretized in advance)
- Examples are partitioned recursively based on selected attributes
- Test attributes are selected based on a heuristic or statistical measure (e.g., **information gain**)



Classification : Decision Tree

ID3 (Iterative Dichotomiser 3) is a classic algorithm used to construct a Decision Tree, primarily for classification tasks. It was developed by Ross Quinlan and serves as one of the earliest algorithms in Decision Tree-based models.

- ID3 aims to create a Decision Tree by selecting the most **significant feature** at each step that best separates the data for classification.
- It does this by using **information gain** (based on entropy) as the criterion to determine the best feature to split on.

Conditions for stopping partitioning:

- All records have the same target class (pure leaf node).
- No more features to split on (terminate as a leaf node).
- Reaching a specified maximum depth or minimum samples threshold to prevent overfitting.

Classification : Decision Tree

Attribute Selection Measure:
Information Gain

D = set of data instances

$|D|$ = number of data instances (14)

p_i = the probability that an arbitrary tuple in D belongs to class C_i

$$p_i = \frac{|C_{i,D}|}{|D|}$$

Example: class yes is 1

$$p_1 = \frac{|C_{1,D}|}{|D|} = \frac{9}{14}$$

$$p_2 = \frac{|C_{2,D}|}{|D|} = \frac{5}{14}$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Expected information (entropy) needed to classify a tuple in D:

$$Info(D) = - \sum_{i=1}^m p_i \log_2 p_i$$

For the given data

$$Info(D) = - \sum_{i=1}^2 p_i \log_2 p_i$$

$$= -p_1 \log_2 p_1 - p_2 \log_2 p_2$$

$$= -\frac{9}{14} \log \frac{9}{14} - \frac{5}{14} \log \frac{5}{14} = 0.940$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information needed (after using A to split D into v partitions)

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Example

$$Info_{age}(D) = \sum_{j=1}^3 \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{5}{14} \times I(2,3) + \frac{4}{14} \times I(4,0) + \frac{5}{14} \times I(2,3) = 0.694$$

$\frac{5}{14} \times I(2,3)$ means “age ≤ 30 ” has 5 out of 14 samples, with 2 yes’es and 3 no’s.

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
>40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information needed (after using A to split D into v partitions)

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Example

$$Info_{age}(D) = \sum_{j=1}^3 \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{5}{14} \times I(2,3) + \frac{4}{14} \times I(4,0) + \frac{5}{14} \times I(2,3) = 0.694$$

$\frac{5}{14} \times I(2,3)$ means “age ≤30” has 5 out of 14 samples, with 2 yes’es and 3 no’s.

age	income	student	credit_rating	buys_computer
≤30	high	no	fair	no
≤30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
≤30	medium	no	fair	no
≤30	low	yes	fair	yes
>40	medium	yes	fair	yes
≤30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information needed (after using A to split D into v partitions)

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Example

$$Info_{age}(D) = \sum_{j=1}^3 \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{5}{14} \times I(2,3) + \frac{4}{14} \times I(4,0) + \frac{5}{14} \times I(3,2) = 0.694$$

$\frac{5}{14} \times I(2,3)$ means “age ≤ 30 ” has 5 out of 14 samples, with 2 yes’es and 3 no’s.

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
>40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information needed (after using A to split D into v partitions)

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Similarly

$$Info_{income}(D) = \sum_{j=1}^3 \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{4}{14} \times I(2,2) + \frac{6}{14} \times I(4,2) + \frac{4}{14} \times I(3,1) = 0.911$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information needed (after using A to split D into v partitions)

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Similarly

$$Info_{income}(D) = \sum_{j=1}^3 \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{4}{14} \times I(2,2) + \frac{6}{14} \times I(4,2) + \frac{4}{14} \times I(3,1) = 0.911$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information needed (after using A to split D into v partitions)

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

Similarly

$$Info_{income}(D) = \sum_{j=1}^3 \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{4}{14} \times I(2,2) + \frac{6}{14} \times I(4,2) + \frac{4}{14} \times I(3,1) = 0.911$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

$$Gain(age) = Info(D) - Info_{age}(D) \\ = 0.940 - 0.694 = 0.246$$

$$Gain(income) \\ = Info(D) - Info_{income}(D) \\ = 0.940 - 0.911 = 0.029$$

Similarly,

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246$$

$$\begin{aligned} Gain(income) &= Info(D) - Info_{income}(D) \\ &= 0.940 - 0.911 = 0.029 \end{aligned}$$

Similarly,

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

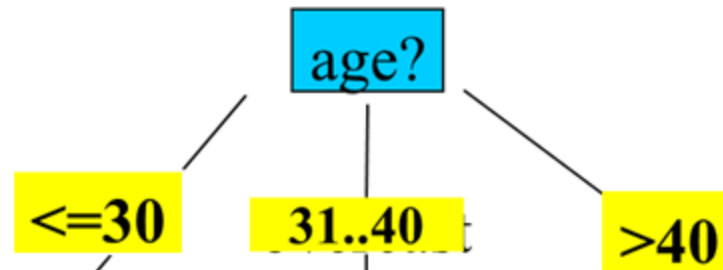


age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

So, the first split is on the **age** attribute

Next, repeat the same process with the other nodes with smaller dataset.



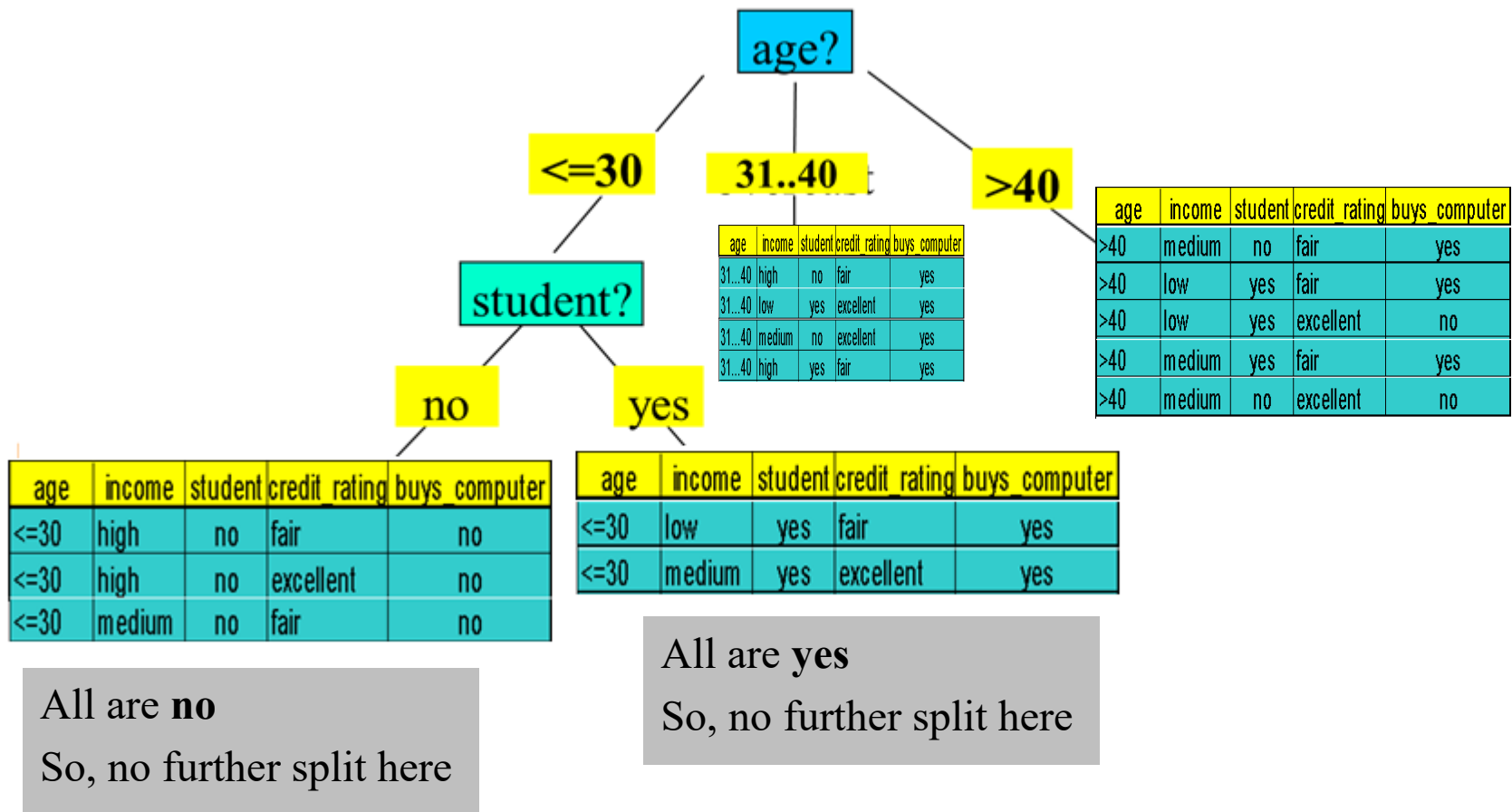
age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
<=30	medium	yes	excellent	yes

age	income	student	credit_rating	buys_computer
31...40	high	no	fair	yes
31...40	low	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes

age	income	student	credit_rating	buys_computer
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
>40	medium	yes	fair	yes
>40	medium	no	excellent	no

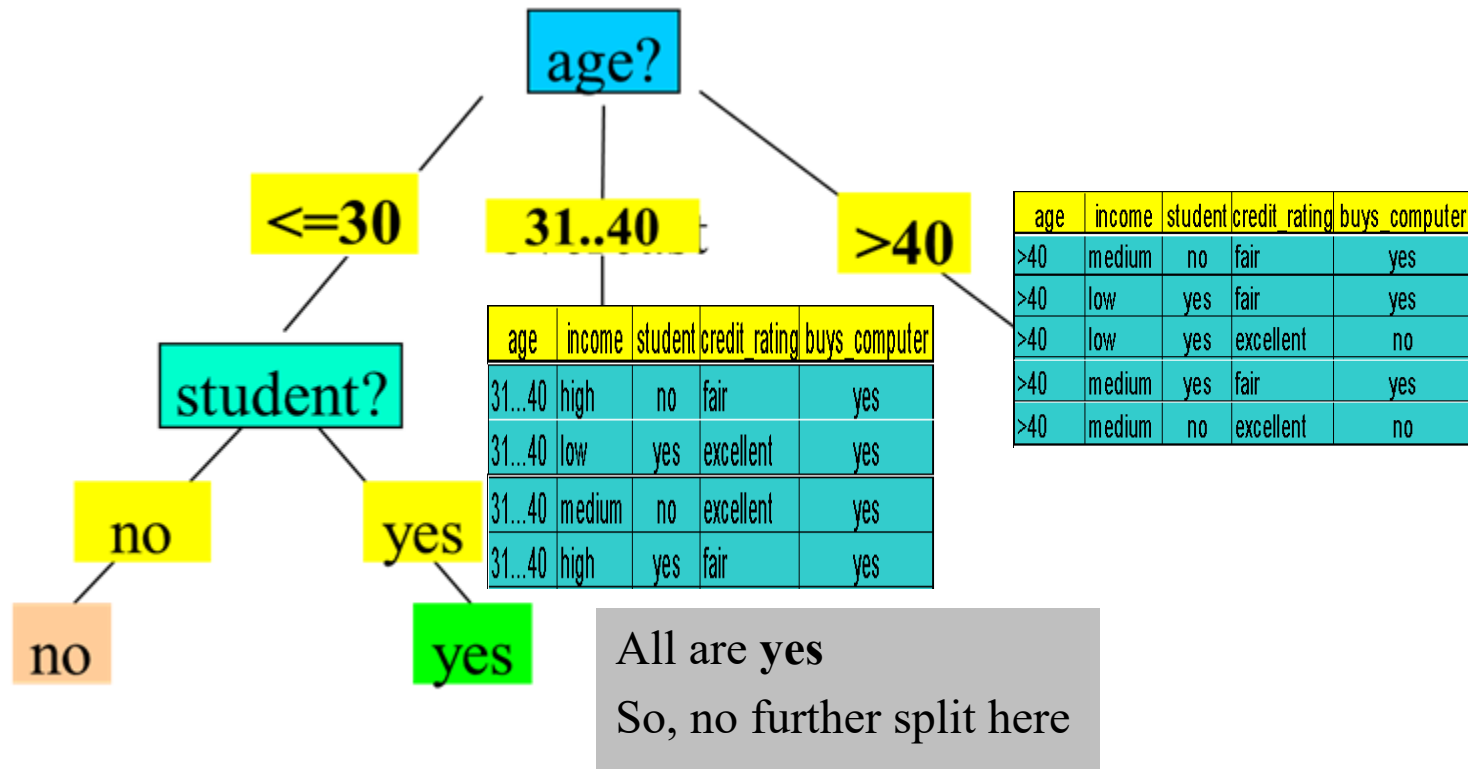
Classification : Decision Tree

Next split is on **student** at node ≤ 30

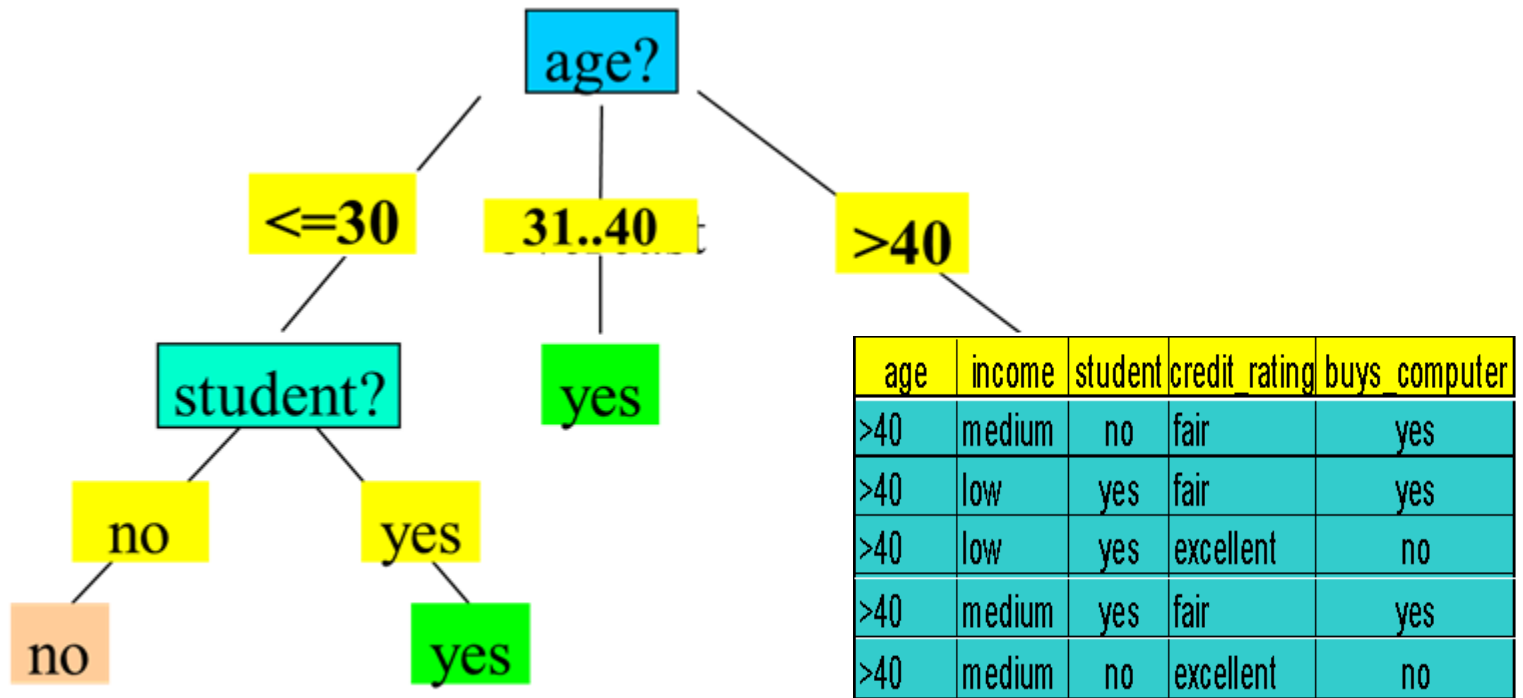


Classification : Decision Tree

No split is required at node **31..40**

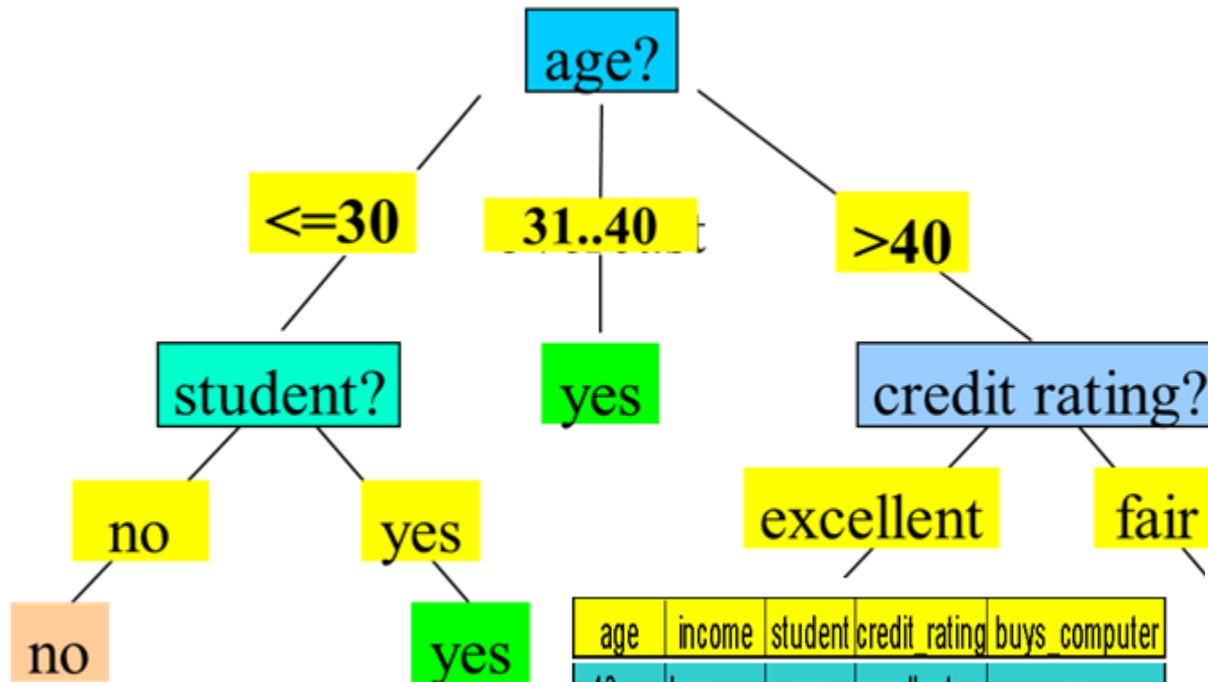


Classification : Decision Tree



Classification : Decision Tree

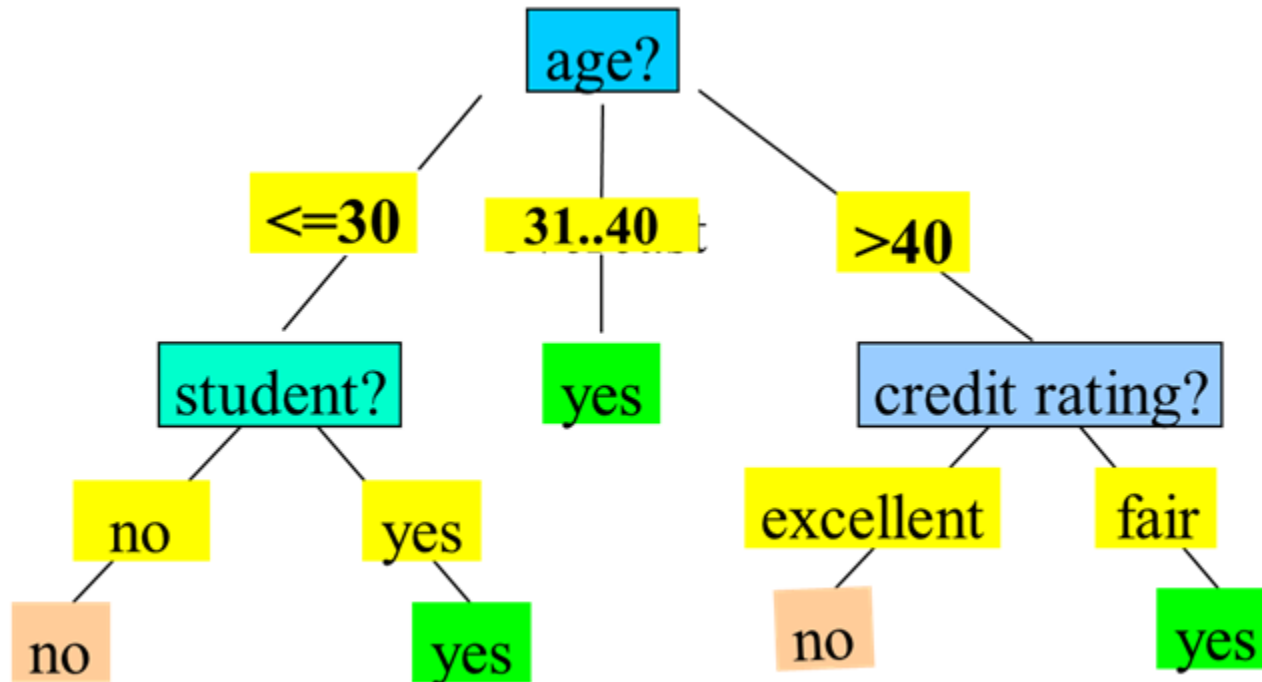
Next split is on **credit_rating** at node **>40**



age	income	student	credit rating	buys computer
>40	low	yes	excellent	no
>40	medium	no	excellent	no

age	income	student	credit_rating	buys_computer
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	medium	yes	fair	yes

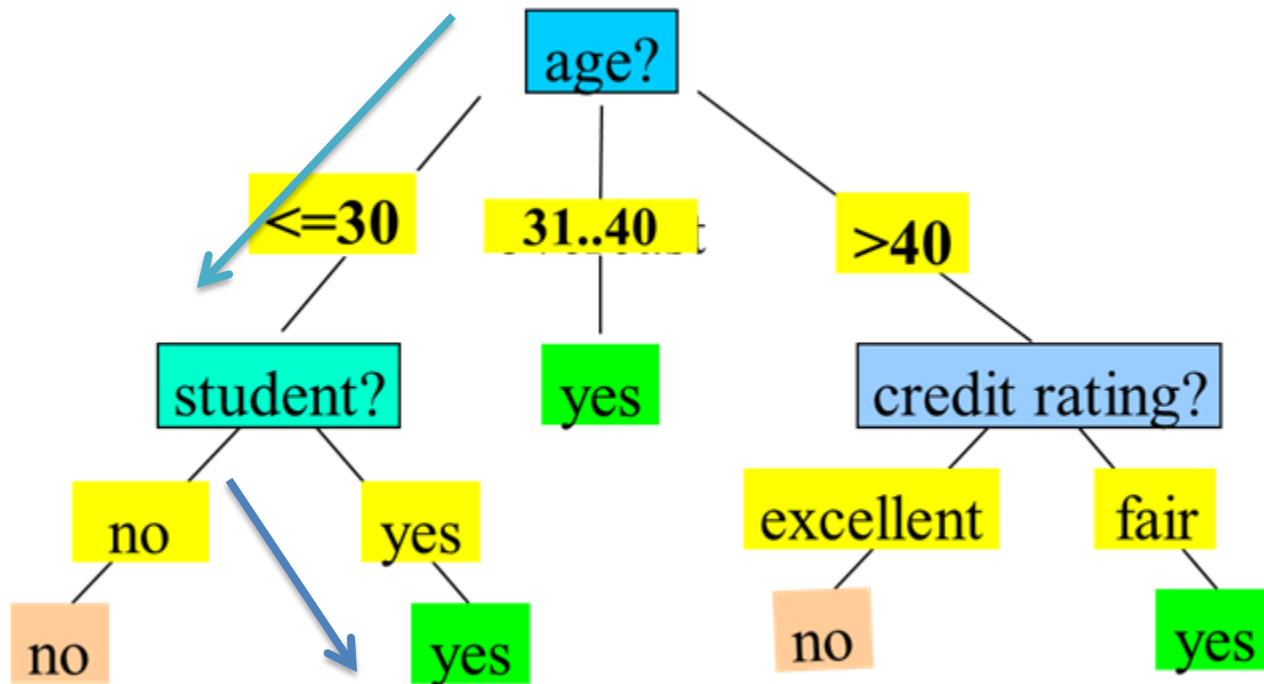
Classification : Decision Tree



Classification : Decision Tree

Classify a new instance

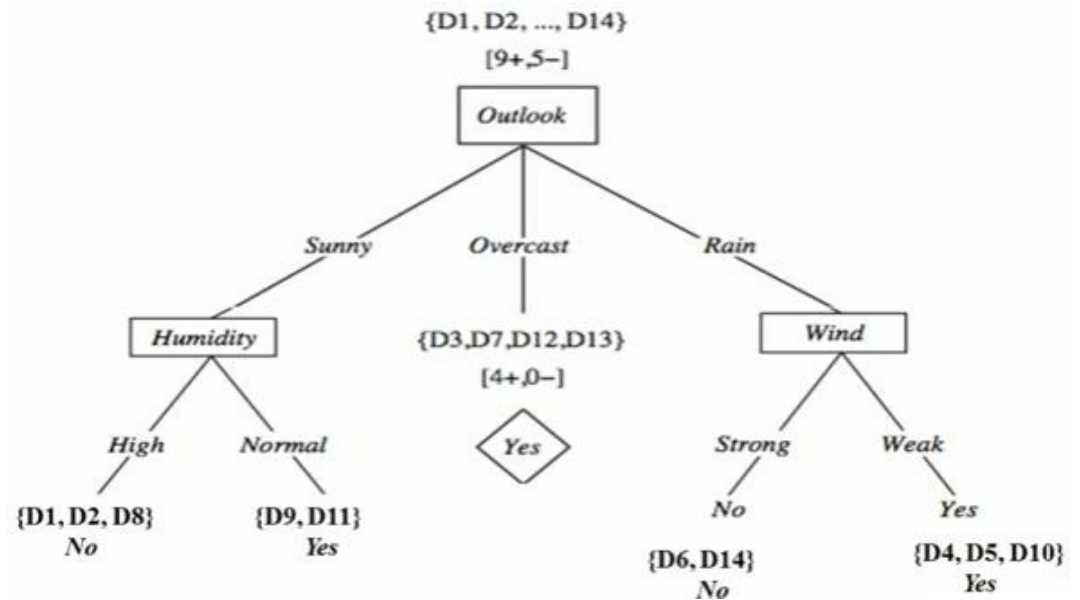
(age: ≤ 30 , income: low, student: yes, credit_rating: fair) = buys_computer?



Classification : Decision Tree

Exercise

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



Classification : Decision Tree

Code

https://colab.research.google.com/drive/1YsHFA55DZ5iQBNq7S11qoS-_WVYtxUYH?usp=sharing

Classification : Decision Tree

Computing Information-Gain for Continuous-Valued Attributes

- Let attribute **A** be a continuous-valued attribute
- Must determine the best split point for **A**
 - Sort the value **A** in increasing order
 - Typically, the midpoint between each pair of adjacent values is considered as a possible split point
 - $(a_i + a_{i+1})/2$ is the midpoint between the values of a_i and a_{i+1}
 - The point with the minimum expected information requirement for **A** is selected as the split-point for **A**
- Split:
 - **D1** is the set of tuples in **D** satisfying $A \leq \text{split-point}$, and **D2** is the set of tuples in **D** satisfying $A > \text{split-point}$

Classification : Decision Tree

Gain Ratio for Attribute Selection (C4.5)

- Information gain measure is biased towards attributes with many values
- C4.5** (a successor of ID3) uses gain ratio to overcome the problem (normalization to information gain)

- $\text{GainRatio}(A) = \text{Gain}(A) / \text{SplitInfo}(A)$

$$\text{SplitInfo}_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

$$\text{SplitInfo}_{\text{income}}(D) = -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) = 1.557$$

- $\text{gain_ratio}(\text{income}) = 0.029 / 1.557 = 0.019$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Classification : Decision Tree

Gini Index (CART-Classification and Regression Trees)

Information Gain and Gain Ratio can sometimes exhibit a bias toward features with many unique values. For example, if a feature has many unique categories, Information Gain might prefer it because it creates more distinct subsets.

To address this, **Gain Ratio** (an adaptation of Information Gain) is used in algorithms like C4.5 to counter this bias, but this additional step adds complexity.

Gini Index, on the other hand, is less sensitive to the number of categories in a feature, reducing the need for an additional adjustment like Gain Ratio.

For a node S with C possible classes, the Gini Index is calculated as:

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2$$

For a split into two nodes, S_1 and S_2 :

$$\text{Gini}_{\text{split}} = \frac{|S_1|}{|S|} \text{Gini}(S_1) + \frac{|S_2|}{|S|} \text{Gini}(S_2)$$

Classification : Decision Tree

Overfitting in Decision Trees

- Overfitting happens when a Decision Tree model becomes too complex, fitting too closely to the noise and specific details of the training data, resulting in poor generalization to new data.
- Signs of Overfitting: A decision tree that overfits will often have a **high accuracy on the training set but performs poorly on the test set**.
- Causes of Overfitting:
 - Allowing the tree to grow too deep, with many nodes and branches.
 - Having very specific branches (splits) that capture the peculiarities of the training data rather than general patterns.
- Consequences: Overfitting leads to a lack of model generalization, where the model captures random fluctuations instead of meaningful trends.

Classification : Decision Tree

Pre-Pruning (Early Stopping):

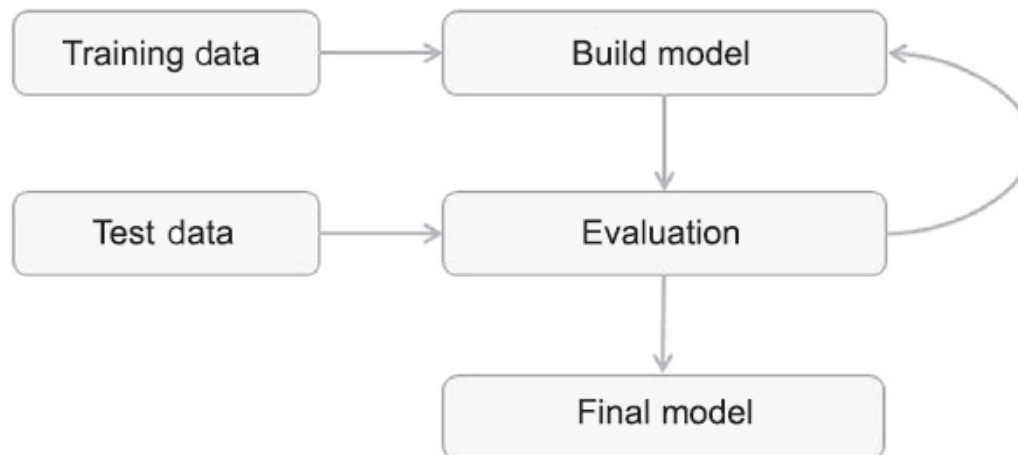
This approach stops the tree from growing once it meets certain conditions, such as reaching a maximum depth, minimum number of samples per leaf, or minimum information gain threshold. This prevents the tree from developing complex branches that might lead to

Post-Pruning (Cost Complexity Pruning):

This technique first grows the tree to its full depth, then prunes back branches that provide minimal value. Post-pruning can be based on measures like cross-validation to determine the optimal structure that minimizes errors on new data.

Model Evaluation

Model evaluation metrics are used to measure the quality and performance of your machine learning models. The choice of metric depends on the type of problem you're solving, primarily **classification** or **regression**.



Performance Measure in Classification

After building a model with a learning algorithm using **the training set**, how can you determine its effectiveness?

The test set is used for this evaluation. Since the test set includes examples the learning algorithm hasn't seen before, good performance on predicting these labels suggests that the model generalizes well—in other words, it's a strong model.

Machine learning specialists use formal **metrics** to evaluate model performance. For classification, the most widely used metrics and tools to assess the classification model are:

- confusion matrix,
- accuracy,
- precision/recall

Performance Measure in Classification

Confusion Matrix

The confusion matrix is a table that shows how well a classification model predicts examples across different classes. One axis represents the model's predicted labels, and the other axis shows the actual labels.

In binary classification, with two classes (for example, “spam” and “not_spam”), this matrix provides a summary of the model's performance.

	Spam (predicted)	Not_spam (predicted)
Spam (actual)	23 (TP)	1 (FN)
Not_spam (actual)	12 (FP)	556 (TN)

Performance Measure in Classification

Confusion Matrix

	Spam (predicted)	Not_spam (predicted)
Spam (actual)	23 (TP)	1 (FN)
Not_spam (actual)	12 (FP)	556 (TN)

The above confusion matrix shows that of the **24 examples** that actually were **spam**, the model correctly classified **23 as spam**. In this case, we say that we have 23 true positives or **TP = 23**. The model incorrectly classified 1 example as not_spam. In this case, we have 1 false negative, or **FN = 1**.

Similarly, of **568 examples** that actually were not spam, 556 were correctly classified (556 true negatives or **TN = 556**), and 12 were incorrectly classified (12 false positives, **FP = 12**).

Performance Measure in Classification

Confusion Matrix

The confusion matrix for **multiclass classification** has as many rows and columns as there are different classes. It can help you to determine mistake patterns.

For example, a confusion matrix could reveal that a model trained to recognize different species of animals tends to mistakenly predict “cat” instead of “panther,” or “mouse” instead of “rat.”

In this case, you can decide to **add more labeled examples** of these species to help the learning algorithm to “see” the difference between them.

Alternatively, you might **add additional features** the learning algorithm can use to build a model that would better distinguish between these species.

Performance Measure in Classification

Precision and Recall

The two most frequently used metrics to assess the model are precision and recall.

Precision is the ratio of correct positive predictions to the overall number of positive predictions:

$$\textit{precision} = \frac{TP}{TP + FP}$$

Recall is the ratio of correct positive predictions to the overall number of positive examples in the dataset:

$$\textit{recall} = \frac{TP}{TP + FN}$$

Performance Measure in Classification

Precision and Recall

	Spam (predicted)	Not_spam (predicted)
Spam (actual)	23 (TP)	1 (FN)
Not_spam (actual)	12 (FP)	556 (TN)

$$precision = \frac{TP}{TP+FP} \qquad recall = \frac{TP}{TP+FN}$$

In the case of the spam detection problem, we want to have **high precision** (we want to avoid making mistakes by detecting that a legitimate message is spam) and we are ready to tolerate **lower recall** (we tolerate some spam messages in our inbox).

Performance Measure in Classification

Precision and Recall

Even if precision and recall are defined for the **binary classification** case, you can always use it to assess a multiclass classification model.

To do that, first select a class for which you want to assess these metrics. Then you consider all examples of the selected class as **positives** and all examples of the remaining classes as **negatives**.

F1 Score

The F1 Score is a performance metric used in classification tasks, especially when dealing with **imbalanced datasets**. It is the harmonic mean of **precision** and **recall**, balancing both metrics into a single value.

$$F1 = \frac{2 \times \textit{precision} \times \textit{recall}}{\textit{precision} + \textit{recall}}$$

Performance Measure in Classification

Accuracy

Accuracy is given by the number of correctly classified examples divided by the total number of classified examples. In terms of the confusion matrix, it is given by:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy is a useful metric when errors in predicting all classes are equally important. In case of the spam/not spam, this may not be the case.

For example, you would tolerate false positives less than false negatives. A false positive in spam detection is the situation in which your friend sends you an email, but the model labels it as spam and doesn't show you. On the other hand, the false negative is less of a problem: if your model doesn't detect a small percentage of spam messages, it's not a big deal.

Model Evaluation: Classification Metrics

F1-Score: The harmonic mean of Precision and Recall. It provides a single score that balances both metrics.

- Formula: $2 * (Precision * Recall) / (Precision + Recall)$
- Example: Useful when you need to find a balance between minimizing false positives and minimizing false negatives, especially in imbalanced datasets.

Model Evaluation: Regression Metrics

These are used when the goal is to predict a continuous numerical value (e.g., a house price, a person's age).

- **Mean Absolute Error (MAE):** The average of the absolute differences between the predicted and actual values.
 - Example: If you are predicting house prices and your MAE is \$10,000, it means that, on average, your model's prediction is off by \$10,000. It's easy to interpret because it's in the same units as the output.
- **Mean Squared Error (MSE):** The average of the squared differences between the predicted and actual values.
 - Example: Similar to MAE but punishes larger errors more heavily because it squares the difference. If your model predicts a price that is off by \$50,000, the squared error is huge, which significantly raises the MSE.

Model Evaluation: Regression Metrics

Root Mean Squared Error (RMSE): The square root of the MSE.

- Example: This is often preferred over MSE because taking the square root brings the metric back into the same units as the original target variable (e.g., dollars for house prices), making it more interpretable. It still penalizes large errors more than MAE.

R-Squared (R^2): Also known as the coefficient of determination. It represents the proportion of the variance in the dependent variable that is predictable from the independent variables.

- Interpretation: It ranges from 0 to 1. An R^2 of 0.82 means that 82% of the variance in the output variable can be explained by the model's inputs. A higher R^2 generally indicates a better fit.